TASK 24: MERGE AND JOIN DATAFRAMES

Project objective

Combine related datasets using Pandas and learn inner, left, right, and outer joins. The project also demonstrates multi-table joins, unmatched-record checks, duplicate-key behavior, relationship validation, and exporting results.

Deliverables

This package contains a complete PDF guide, executable Python script, explanatory Jupyter Notebook, three input CSV files, join-result CSV files, unmatched-record reports, and a README.

1. Concepts and terminology

DataFrame: a two-dimensional table of rows and columns in Pandas. **Key:** a column used to match records across tables. **Merge:** an operation that combines rows from two DataFrames according to their keys. **NaN:** a missing value in the result. **Cardinality:** the relationship between key occurrences, such as one-to-one, one-to-many, or many-to-many.

2. Sample datasets

customers.csv has one row per customer; orders.csv has transaction rows; payments.csv records payment status by OrderID. The data intentionally includes CustomerID 106 in orders but not customers, customers 103 and 105 with no orders, orders without payment rows, and Payment OrderID 5006 with no corresponding order.

Customers

CustomerID	CustomerName	City
101	Aarav Sharma	Delhi
102	Priya Singh	Lucknow
103	Rahul Verma	Kanpur
104	Sneha Gupta	Noida
105	Kabir Khan	Fatehpur

Orders

OrderID	CustomerID	Amount
5001	101	2500
5002	102	1800
5003	102	3200
5004	104	950
5005	106	1200

Payments

PaymentID	OrderID	PaymentStatus
9001	5001	Paid

PaymentID	OrderID	PaymentStatus
9002	5002	Pending
9003	5004	Paid
9004	5006	Failed

3. Import Pandas and inspect the data

```
import pandas as pd
customers = pd.read_csv("customers.csv")
orders = pd.read_csv("orders.csv")
payments = pd.read_csv("payments.csv")

print(customers.shape)  # (rows, columns)
print(customers.dtypes) # data type of each column
display(customers.head())
```

`read_csv()` loads CSV data into a DataFrame. `shape` returns row and column counts, `dtypes` shows column data types, and `head()` previews the first five rows. Confirm that join keys are present and have compatible types.

4. Inner join

```
inner_join = customers.merge(
    orders, on="CustomerID", how="inner"
)
display(inner_join)
```

`how='inner'` keeps keys that match in both tables. With this sample, CustomerIDs 101, 102, and 104 match. CustomerID 102 appears twice because it has two orders. Unmatched customers and the order for CustomerID 106 are excluded.

5. Left join

```
left_join = customers.merge(
    orders, on="CustomerID", how="left"
)
display(left_join)
```

`how='left'` keeps every row from customers and adds matching order fields. Customers 103 and 105 remain, with NaN in order columns. Use this when the left table is the master list that must be preserved.

6. Right join

```
right_join = customers.merge(
    orders, on="CustomerID", how="right"
)
display(right_join)
```

`how='right'` keeps every order. Order 5005 remains even though CustomerID 106 is missing from customers, so customer attributes are NaN. It can also be expressed by swapping the DataFrames and using a left join.

7. Outer join

```
outer_join = customers.merge(
    orders, on="CustomerID", how="outer"
)
display(outer_join)
```

`how='outer'` keeps all records from both sides. It is useful for reconciliation because it exposes customer rows without orders and order rows without known customers.

8. Merge indicator and unmatched records

```
left_check = customers.merge(
    orders, on="CustomerID", how="left", indicator=True
)
```

```
print(left_check["_merge"].value_counts())
customers_without_orders = left_check[
    left_check["_merge"] == "left_only"
]
```

`indicator=True` adds `_merge`: `both` means a match, `left_only` means the row came only from the left side, and `right_only` means it came only from the right side. For a right join, filter `right_only` to find orders without a known customer.

9. Join orders with payments

```
orders_with_payments = orders.merge(
    payments, on="OrderID", how="left",
    indicator="PaymentMatch"
)
display(orders_with_payments)
```

Use OrderID because that is the relationship between orders and payments. A left join retains all orders. An order with no matching payment row is not necessarily unpaid; the supplied data may simply be incomplete.

10. Combine three DataFrames

```
final_dataset = (
    customers
    .merge(orders, on="CustomerID", how="outer")
    .merge(payments, on="OrderID", how="left")
)
display(final_dataset)
```

First combine customer and order information using CustomerID. Then attach payment information using OrderID. The selected join types determine which records survive. Because payment data is left-joined, payment-only OrderID 5006 does not appear in this final dataset; inspect payment-only records separately if needed.

11. Duplicate keys and relationship cardinality

```
print(customers["CustomerID"].is_unique) # True
print(orders["CustomerID"].is_unique)    # False

orders_per_customer = orders.groupby(
    "CustomerID", as_index=False
).size().rename(columns={"size": "OrderCount"})
display(orders_per_customer)
```

CustomerID is unique in customers but repeats in orders, a one-to-many relationship. Duplicate keys are not inherently bad. If both sides have repeated keys, a many-to-many merge can multiply rows. Always compare row counts and ensure that repeated keys match the business meaning.

12. Validate expected relationships

```
checked = customers.merge(
    orders, on="CustomerID", how="left",
    validate="one_to_many"
)
```

`validate='one_to_many'` verifies that left-side keys are unique and right-side keys may repeat. Other options include `one_to_one`, `one_to_many`, `many_to_one`, and `many_to_many`. Validation catches assumptions that do not match the data.

13. Different key column names

```
# Example if the right table used ClientID instead:
# customers.merge(
#     orders.rename(columns={"CustomerID": "ClientID"}),
#     left_on="CustomerID", right_on="ClientID",
#     how="left"
# )
```

Use `left_on` and `right_on` when the key columns have different names. The values must still represent the same identifier and use compatible data types.

14. Export results

```
final_dataset.to_csv(
    "final_combined_dataset.csv", index=False
)
```

`to_csv()` writes a DataFrame to a CSV file. `index=False` prevents Pandas from saving the row index as an extra column. The ZIP includes separate CSVs for each join and the final dataset.

15. Common problems and best practices

```
# Check key uniqueness and nulls
print(customers["CustomerID"].is_unique)
print(customers["CustomerID"].isna().sum())

# Resolve same-named non-key columns using suffixes
# df1.merge(df2, on="ID", suffixes=("_left", "_right"))
```

Check the join key, data types, missing IDs, duplicate IDs, and row counts. Use suffixes for duplicate non-key column names. Do not interpret NaN as a business status without confirming the source. Investigate null keys because their merge behavior may differ from assumptions based on SQL.

16. Join type comparison

Join type	Rows retained	Typical use
Inner	Only matching keys	Find records present in both sources
Left	All left rows	Keep a complete master list
Right	All right rows	Keep all transactions from right table
Outer	All rows from both	Reconciliation and gap detection

17. Interview questions and answers

Q1. What is merge in Pandas?

`merge()` combines DataFrames based on common columns or indexes, similar to SQL joins.

Q2. What is the difference between inner and left joins?

Inner keeps matching keys only; left preserves all rows from the left DataFrame.

Q3. Why do duplicate keys create unexpected results?

Each matching occurrence can pair with every matching row on the other side, multiplying output rows and potentially inflating aggregates.

Q4. How can you identify unmatched records?

Use `indicator=True` and filter `_merge` for `left_only` or `right_only`.

Q5. How do you merge using differently named keys?

Use `left_on=` and `right_on=` instead of `on=`.

Q6. What does `validate='one_to_many'` do?

It checks that the left keys are unique while right-side keys may repeat.

Q7. What does NaN mean after a join?

No value was available for that output field; it may indicate no match or missing source data.

Q8. Why save separate join outputs?

Separate outputs make comparisons, debugging, quality checks, and task submission easier.